Set-A

1. **Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.**
   The first node is considered odd, and the second node is even, and so on.
   Note that the relative order inside both the even and odd groups should remain as it was in the input.
   Input: [1,2,3,4,5]
   Output: [1,3,5,2,4]

One of the solution:

```
ListNode* oddEvenList(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return head;
    }

    ListNode* odd = head;
    ListNode* even = head->next;
    ListNode* evenHead = even; // to reconnect later

    while (even != nullptr && even->next != nullptr) {
        odd->next = even->next;
        odd = odd->next;

        even->next = odd->next;
        even = even->next;
    }

    odd->next = evenHead; // attach even list at the end
```

```
    return head;
}
```

****

2. **Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.**

   Input: [1,1,2,3,3]

   Output: [1,2,3]

One of the solution:

```
ListNode* deleteDuplicates(ListNode* head) {
    ListNode* current = head;

    while (current != nullptr && current->next != nullptr) {
        if (current->val == current->next->val) {
            // skip the duplicate node
            ListNode* temp = current->next;
            current->next = current->next->next;
            delete temp; // optional: free memory
        } else {
            current = current->next;
        }
    }

    return head;
}
```

****

3. **Given the head of a singly linked list, return the middle node of the linked list. If there are two middle nodes, return the second middle node.**
   Input: [1,2,3,4,5]
   Output: [3]

One of the solution:

```
ListNode* middleNode(ListNode* head) {
   ListNode* slow = head;
   ListNode* fast = head;

   while (fast != nullptr && fast->next != nullptr) {
      slow = slow->next;       // move 1 step
      fast = fast->next->next;  // move 2 steps
   }

   return slow;  // slow is now at middle
}
```

****

4. **In a linked list of size n, where n is even, the ith node (0-indexed) of the linked list is known as the twin of the (n-1-i)th node, if 0 <= i <= (n / 2) - 1.**
   For example, if n = 4, then node 0 is the twin of node 3, and node 1 is the twin of node 2. These are the only nodes with twins for n = 4.
   The twin sum is defined as the sum of a node and its twin.
   Given the head of a linked list with even length, return the maximum twin sum of the linked list.
   Input: [4,2,2,3]
   Output: 7

One of the solution:

```
int pairSum(ListNode* head) {
   // Step 1: Find middle
```

```
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Step 2: Reverse second half
    ListNode* prev = nullptr;
    ListNode* curr = slow;
    while (curr != nullptr) {
        ListNode* nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    // Step 3: Compute twin sums
    int maxSum = 0;
    ListNode* first = head;
    ListNode* second = prev;  // head of reversed second half
    while (second != nullptr) {
        int sum = first->val + second->val;
        maxSum = max(maxSum, sum);
        first = first->next;
        second = second->next;
    }

    return maxSum;
}
```

****

Set-B

1. **Given the head of a singly linked list and an integer k, split the linked list into k consecutive linked list parts.**
   The length of each part should be as equal as possible: no two parts should have a size differing by more than one. This may lead to some parts being null.
   The parts should be in the order of occurrence in the input list, and parts occurring earlier should always have a size greater than or equal to parts occurring later.
   Return an array of the k parts.

   Input: [1,2,3,4,5,6,7,8,9,10], k = 3

   Output: [[1,2,3,4],[5,6,7],[8,9,10]]

One of the solution:

```
struct ListNode** splitListToParts(struct ListNode* head, int k, int* returnSize) {
    *returnSize = k;
    struct ListNode** result = (struct ListNode**)malloc(sizeof(struct ListNode*) * k);

    // Step 1: Find length n
    int n = 0;
    struct ListNode* curr = head;
    while (curr) {
        n++;
        curr = curr->next;
    }

    // Step 2: Calculate base size and extra
    int baseSize = n / k;
    int extra = n % k;

    curr = head;
    for (int i = 0; i < k; i++) {
        if (!curr) {
            result[i] = NULL;  // empty part
            continue;
        }

        result[i] = curr; // head of this part
        int partSize = baseSize + (extra > 0 ? 1 : 0);
```

```
        if (extra > 0) extra--;

        // Traverse this part
        for (int j = 1; j < partSize; j++) {
            if (curr) curr = curr->next;
        }

        // Cut the list
        if (curr) {
            struct ListNode* nextPart = curr->next;
            curr->next = NULL;
            curr = nextPart;
        }
    }

    return result;
}
```

****

2. **Given the head of a linked list and an integer val, remove all the nodes of the linked list that has Node.val == val, and return the new head.**

   Input: [1,2,6,3,4,5,6], val = 6

   Output: [1,2,3,4,5]

One of the solution:

```
struct ListNode* removeElements(struct ListNode* head, int val) {
    // Create a dummy node before head (to handle head deletion easily)
    struct ListNode* dummy = (struct ListNode*)malloc(sizeof(struct ListNode));
    dummy->next = head;

    struct ListNode* prev = dummy;
    struct ListNode* curr = head;

    while (curr) {
```

```
        if (curr->val == val) {
            // Skip the current node
            prev->next = curr->next;
            free(curr);
            curr = prev->next;
        } else {
            prev = curr;
            curr = curr->next;
        }
    }

    struct ListNode* newHead = dummy->next;
    free(dummy);
    return newHead;
}
```

****

3. **You are given the head of a linked list with n nodes.**
   For each node in the list, find the value of the next greater node. That is, for each node, find the value of the first node that is next to it and has a strictly larger value than it.

   Return an integer array answer where answer[i] is the value of the next greater node of the ith node (1-indexed).
   If the ith node does not have a next greater node, set answer[i] = 0.

One of the solution:

```
int* nextLargerNodes(struct ListNode* head, int* returnSize) {
    // Step 1: Store values into an array
    int n = 0;
    struct ListNode* temp = head;
    while (temp) {
        n++;
        temp = temp->next;
    }

    int* values = (int*)malloc(n * sizeof(int));
    temp = head;
```

```
    for (int i = 0; i < n; i++) {
        values[i] = temp->val;
        temp = temp->next;
    }

    // Step 2: Result array
    int* answer = (int*)calloc(n, sizeof(int));

    // Step 3: Monotonic stack (stores indices)
    int* stack = (int*)malloc(n * sizeof(int));
    int top = -1; // stack pointer

    for (int i = 0; i < n; i++) {
        while (top >= 0 && values[i] > values[stack[top]]) {
            answer[stack[top]] = values[i];
            top--;
        }
        stack[++top] = i; // push index
    }

    *returnSize = n;
    free(values);
    free(stack);
    return answer;
}
```

****

4. **Given the pointer to the head node of a doubly linked list, reverse the order of the nodes in place. That is, change the next and prev pointers of the nodes so that the direction of the list is reversed.**

   The initial doubly linked list is: 1 <-> 2 <-> 3 <-> 4

   The reversed doubly linked list is: 4 <-> 3 <-> 2 <-> 1

One of the solution:

```
struct Node* reverseDoublyList(struct Node* head) {
```

```
    if (head == NULL) return NULL;

    struct Node* current = head;
    struct Node* temp = NULL;

    // Traverse and swap next and prev for every node
    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        // Move to the next node (which is prev after swap)
        current = current->prev;
    }

    // After loop, temp will be at the new head's previous node
    if (temp != NULL) {
        head = temp->prev;
    }

    return head;
}
```

****

Set-C

1. **Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.**

   The first node is considered odd, and the second node is even, and so on.

   Note that the relative order inside both the even and odd groups should remain as it was in the input.

   Input: [1,2,3,4,5]

   Output: [1,3,5,2,4]

One of the solution:

```
ListNode* oddEvenList(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return head;
    }

    ListNode* odd = head;
    ListNode* even = head->next;
    ListNode* evenHead = even; // to reconnect later

    while (even != nullptr && even->next != nullptr) {
        odd->next = even->next;
        odd = odd->next;

        even->next = odd->next;
        even = even->next;
    }

    odd->next = evenHead; // attach even list at the end
    return head;
}
```

****

2. **Given the head of a linked list, rotate the list to the right by k places.**

   Input: [1,2,3,4,5], k = 2
   Output: [4,5,1,2,3]

One of the solution:

```
struct ListNode* rotateRight(struct ListNode* head, int k) {
    if (!head || !head->next || k == 0) return head;

    // Step 1: Find length and tail
    struct ListNode* tail = head;
    int n = 1;
    while (tail->next) {
        tail = tail->next;
        n++;
    }

    // Step 2: Adjust k
    k = k % n;
    if (k == 0) return head;

    // Step 3: Connect tail to head (circular list)
    tail->next = head;

    // Step 4: Find new tail (n - k steps from start)
    int stepsToNewTail = n - k;
    struct ListNode* newTail = head;
    for (int i = 1; i < stepsToNewTail; i++) {
        newTail = newTail->next;
    }

    // Step 5: Break the circle
    struct ListNode* newHead = newTail->next;
    newTail->next = NULL;

    return newHead;
}
```

****

3. **Given the head of a linked list and an integer val, remove all the nodes of the linked list that has Node.val == val, and return the new head.**

   Input: [1,2,6,3,4,5,6], val = 6

   Output: [1,2,3,4,5]

One of the solution:

```
struct ListNode* removeElements(struct ListNode* head, int val) {
    // Create a dummy node before head (to handle head deletion easily)
    struct ListNode* dummy = (struct ListNode*)malloc(sizeof(struct ListNode));
    dummy->next = head;

    struct ListNode* prev = dummy;
    struct ListNode* curr = head;

    while (curr) {
        if (curr->val == val) {
            // Skip the current node
            prev->next = curr->next;
            free(curr);
            curr = prev->next;
        } else {
            prev = curr;
            curr = curr->next;
        }
    }

    struct ListNode* newHead = dummy->next;
    free(dummy);
    return newHead;
}
```

****

4. **You are given the heads of two sorted linked lists list1 and list2.**

   Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

   Return the head of the merged linked list.

   Input: [1,2,4], list2 = [1,3,4]

   Output: [1,1,2,3,4,4]

One of the solution:

```
struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
    // Dummy node
    struct ListNode dummy;
    struct ListNode* tail = &dummy;
    dummy.next = NULL;

    // Compare and merge
    while (list1 && list2) {
        if (list1->val <= list2->val) {
            tail->next = list1;
            list1 = list1->next;
        } else {
            tail->next = list2;
            list2 = list2->next;
        }
        tail = tail->next;
    }

    // Attach remaining nodes
    if (list1) tail->next = list1;
    if (list2) tail->next = list2;

    return dummy.next;
}
```

****

Set-D

1. **In a linked list of size n, where n is even, the ith node (0-indexed) of the linked list is known as the twin of the (n-1-i)th node, if 0 <= i <= (n / 2) - 1.**

   For example, if n = 4, then node 0 is the twin of node 3, and node 1 is the twin of node 2. These are the only nodes with twins for n = 4.

   The twin sum is defined as the sum of a node and its twin.

   Given the head of a linked list with even length, return the maximum twin sum of the linked list.

   Input: [4,2,2,3]

   Output: 7

One of the solution:

```
int pairSum(ListNode* head) {
   // Step 1: Find middle
   ListNode* slow = head;
   ListNode* fast = head;
   while (fast != nullptr && fast->next != nullptr) {
      slow = slow->next;
      fast = fast->next->next;
   }

   // Step 2: Reverse second half
   ListNode* prev = nullptr;
   ListNode* curr = slow;
   while (curr != nullptr) {
      ListNode* nextNode = curr->next;
      curr->next = prev;
      prev = curr;
      curr = nextNode;
```

```
    }

    // Step 3: Compute twin sums
    int maxSum = 0;
    ListNode* first = head;
    ListNode* second = prev;  // head of reversed second half
    while (second != nullptr) {
        int sum = first->val + second->val;
        maxSum = max(maxSum, sum);
        first = first->next;
        second = second->next;
    }

    return maxSum;
}
```

****

2. **Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.**
   Input: [1,1,2,3,3]
   Output: [1,2,3]

One of the solution:

```
struct ListNode* deleteDuplicates(struct ListNode* head) {
    struct ListNode* current = head;

    while (current && current->next) {
        if (current->val == current->next->val) {
            // Remove duplicate node
            struct ListNode* temp = current->next;
            current->next = current->next->next;
```

```
            free(temp);
        } else {
            current = current->next;
        }
    }

    return head;
}
```

****

3. **Given the head of a linked list, rotate the list to the right by k places.**

Input: [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]

One of the solution:

```
struct ListNode* rotateRight(struct ListNode* head, int k) {
    if (!head || !head->next || k == 0) return head;

    // Step 1: Find length and tail
    struct ListNode* tail = head;
    int n = 1;
    while (tail->next) {
        tail = tail->next;
        n++;
    }

    // Step 2: Adjust k
    k = k % n;
    if (k == 0) return head;

    // Step 3: Connect tail to head (circular list)
    tail->next = head;

    // Step 4: Find new tail (n - k steps from start)
    int stepsToNewTail = n - k;
    struct ListNode* newTail = head;
    for (int i = 1; i < stepsToNewTail; i++) {
        newTail = newTail->next;
    }
```

```
    // Step 5: Break the circle
    struct ListNode* newHead = newTail->next;
    newTail->next = NULL;

    return newHead;
}
```

****

4. **You are given the head of a linked list with n nodes.**
   For each node in the list, find the value of the next greater node. That is, for each node, find the value of the first node that is next to it and has a strictly larger value than it.

   Return an integer array answer where answer[i] is the value of the next greater node of the ith node (1-indexed).
   If the ith node does not have a next greater node, set answer[i] = 0.

One of the solution:

```
int* nextLargerNodes(struct ListNode* head, int* returnSize) {
    // Step 1: Store values into an array
    int n = 0;
    struct ListNode* temp = head;
    while (temp) {
        n++;
        temp = temp->next;
    }

    int* values = (int*)malloc(n * sizeof(int));
    temp = head;
    for (int i = 0; i < n; i++) {
        values[i] = temp->val;
        temp = temp->next;
    }

    // Step 2: Result array
    int* answer = (int*)calloc(n, sizeof(int));
```

```
    // Step 3: Monotonic stack (stores indices)
    int* stack = (int*)malloc(n * sizeof(int));
    int top = -1; // stack pointer

    for (int i = 0; i < n; i++) {
        while (top >= 0 && values[i] > values[stack[top]]) {
            answer[stack[top]] = values[i];
            top--;
        }
        stack[++top] = i; // push index
    }

    *returnSize = n;
    free(values);
    free(stack);
    return answer;
}
```

****